

SOLID PRINCIPI

Uvod

Ovaj dokument daje pregled usklađenosti priloženog UML dijagrama klasa sa SOLID principima. Analiza je bazirana na modelu koji sadrži klase **Korisnik**, **Putnik**, **Destinacija**, **Paket**, **Rezervacija**, **Plaćanje**, **Notifikacija**, **PlanPutovanja**, **StavkaPlana** i **UslugaPaketa** kao i enumeracije **Uloga**, **StatusPaketa**, **StatusRezervacije**, **MetodaPlaćanja**, **TipNotifikacije**.

1. S - Single Responsibility Principle (Principi dobrog dizajna)

KLASA BI TREBALA IMATI SAMO JEDAN RAZLOG ZA PROMJENU.

Primjena u modelu:

U dijagramu su entitetske klase (npr. **Korisnik**, **Paket**, **Rezervacija**, **Plaćanje**, **Notifikacija**, **PlanPutovanja**) fokusirane na podatke i osnovna pravila. Iako se određena poslovna logika (kreiranje i otkazivanje rezervacije, obrada plaćanja, slanje potvrda) nalazi unutar ovih klasa, ona je i dalje vezana za njihovu osnovnu odgovornost. Zbog toga se može smatrati da **SRP** princip nije značajno narušen, jer svaka klasa i dalje ima jasno definisanu ulogu u okviru sistema.

2. O - Open/Closed Principle (Otvoreno zatvoren princip)

ENTITETI SOFTVERA (KLASE, MODULI, METODE) TREBALI BI BITI OTVORENI ZA NADOGRADNJU, ALI ZATVORENI ZA MODIFIKACIJE.

Primjena u modelu:

Sistem treba biti otvoren za proširenja, ali da se postojeći kod ne mora mijenjati svaki put kad dodamo novu opciju. U prikazanom modelu, proširenja se mogu postići dodavanjem novih metoda ili proširenjem postojećih klasa. Na primjer, dodavanje nove metode plaćanja ili novog tipa notifikacije može zahtijevati izmjene unutar klase **Plaćanje** ili **Notifikacija**. Enumeracije (npr. **MetodaPlaćanja**, **StatusRezervacije**) pomažu da se stanja i vrste jasno definišu i koriste kroz sistem.

3. L - Liskov Substitution Principle (Liskov princip zamjene)

PODTIPOVI MORAJU BITI ZAMJENJIVI NJIHOVIM OSNOVNIM TIPOVIMA.

Primjena u modelu:

U prikazanom dijagramu ne postoji izraženo nasljeđivanje niti hijerarhija podtipova među klasama, pa se **LSP** princip ne može direktno uočiti kroz zamjenjivost podklasa. Većina klasa je modelirana kao zasebni entiteti bez apstraktnih baza ili interfejsa. Zbog toga se može reći da **LSP** princip nije značajno

primijenjen u ovom modelu, ali također nije ni narušen, jer ne postoje odnosi nasljeđivanja u kojima bi moglo doći do njegove povrede.

4. I - Interface Segregation Principle (Princip segregacije interfejsa)

KLIJENTI NE TREBA DA OVISE O METODAMA KOJE NEĆE UPOTREBLJAVATI.

Primjena u modelu:

Bolje je imati više manjih i jasnih interfejsa nego jedan veliki koji sadrži sve. U prikazanom modelu nisu definisani posebni interfejsi, već klase direktno sadrže svoje metode, gdje većina klasa ima jasno definisane odgovornosti. Može reći da ovaj princip nije u potpunosti primijenjen, ali struktura modela i dalje omogućava relativno jasnu podjelu odgovornosti između klasa.

5. D - Dependency Inversion Principle (Princip inverzije ovisnosti)

MODULI VISOKOG NIVOA NE BI TREBALI OVISITI OD MODULA NISKOG NIVOA. OBA BI TREBALA DA OVISE OD APSTRAKCIJA.

MODULI NE BI TREBALI OVISITI OD DETALJA. DETALJI BI TREBALI BITI OVISNI OD APSTRAKCIJA.

Primjena u modelu:

Glavna poslovna logika ne treba direktno zavisiti od konkretnih detalja (npr. tačan način slanja emaila ili način obrade plaćanja), već od apstrakcija. U prikazanom dijagramu nisu eksplicitno definisane apstrakcije poput interfejsa ili apstraktnih klasa, već su zavisnosti ostvarene direktno između klasa (npr. **Rezervacija, Plaćanje, Notifikacija**). Zbog toga sistem u određenoj mjeri odstupa od **DIP** principa, jer viši nivoi direktno zavise od konkretnih implementacija. Ipak, struktura modela je dovoljno jasna da bi se u budućnosti mogle uvesti apstrakcije bez većih izmjena, čime bi se princip dodatno ispoštovao.

Zaključak:

Na osnovu priloženog UML dijagrama klasa, model je u dobroj mjeri usklađen sa **SOLID** principima. Poslovna logika (npr. rezervacije, plaćanja i notifikacije) je većinom smještena unutar odgovarajućih klasa, što je prihvatljivo jer je vezana za njihove osnovne odgovornosti. Ipak, izostanak posebnih apstrakcija (servisa ili interfejsa) znači da određene promjene mogu zahtijevati izmjene postojećih klasa.

Sistem omogućava proširenja (npr. nove metode plaćanja ili tipove notifikacija). Enumeracije doprinose jasnoći i standardizaciji tipova i stanja u sistemu. Uvođenjem dodatnih apstrakcija (npr. interfejsa ili servisnog sloja), model bi mogao dodatno unaprijediti usklađenost sa **OCP** i **DIP** principima i smanjiti međuzavisnosti između komponenti.